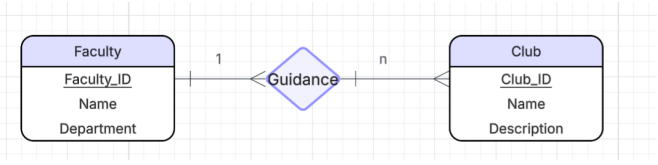
Enterprise Description

1. Background Information: Our enterprise is a student event management system. It operates within a university campus environment and focuses specifically on managing the extracurricular activities of student life. While colleges already have a system for academic records for course enrollment and grades, there remains a significant need for a professional system to manage student bodies' extracurricular activities. Our system mainly serves student bodies including student organizations, student membership, faculty oversight, and the planning and execution of events held across the campus.
2. Operation of the Enterprise:
   a. Club Governance: Each club is required to appoint a faculty advisor who is responsible for providing oversight, approving planned activities, and ensuring that the organization operates in accordance with university policies.
   b. Student Advising: The faculty assist students in exploring different organizations and identifying groups that align with their interests. Through this advisory relationship, students receive personalized support as they engage in campus life.
   c. Club Membership Management: The system records and maintains information on students who join clubs. When a student becomes a member, a corresponding membership record is created, linking the student to the organization and assigning them a specific role.
   d. Event Management: (1) Member Events: Internal events intended exclusively for club members, including meetings, workshops, and planning sessions. (2) Public Events: Campus activities open to all students, such as fairs, cultural programs, or guest speaker sessions. The system supports event scheduling, room reservations, participant tracking, and enforcement of specific event access rules.
3. The Need for the Database: A database is essential for managing campus organizations and events. For instance, LSU admitted 8,153 freshmen in Fall 2025, which is 3.8% higher from the previous year, bringing the total student population to 37,354. As enrollment continues to rise, the scale of student involvement increases proportionally. Relying on paper-based methods to manually track every detail becomes impractical and prone to mistakes. Manual systems can't prevent duplicated records, scheduling conflicts, lost information, or inconsistencies across student organizations. Using a database is a necessary step to maintain operational efficiency, accountability, and improve the overall quality of student engagement services.

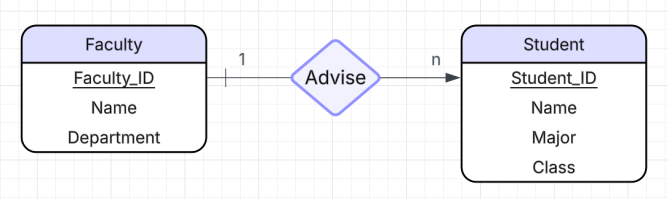
E-R Diagram

Sub-diagram:

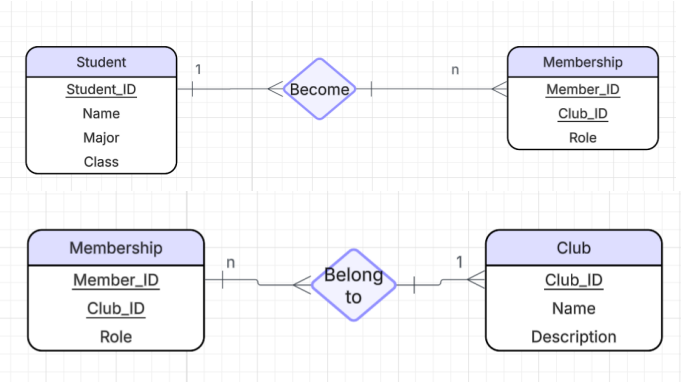
(1) Club Governance



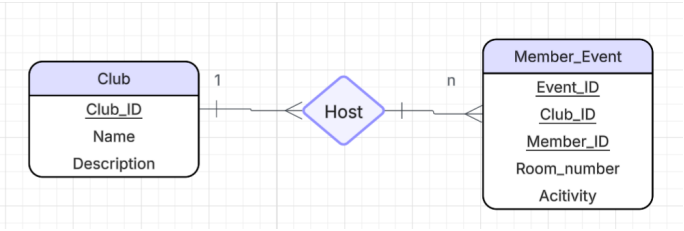
(2) Student and Faculty Advising

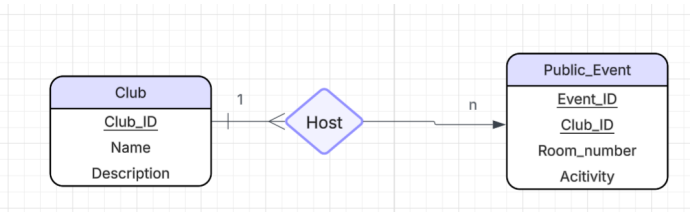


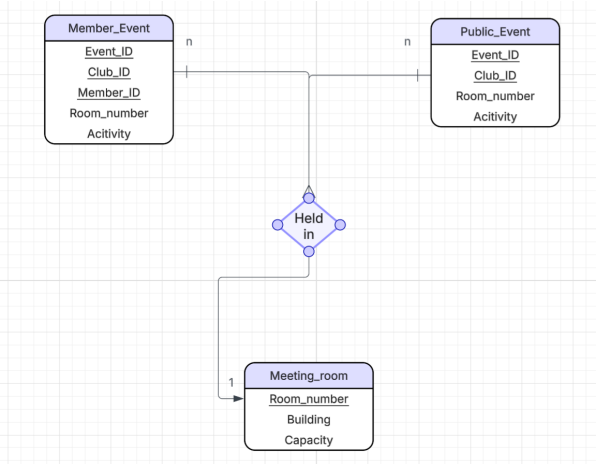
(3) Club Membership



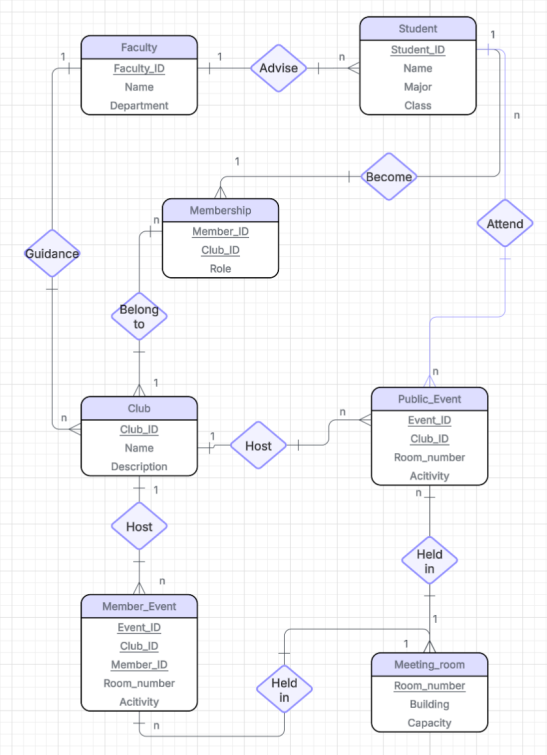
(4) Event Schedule







Master-Diagram:



Relationship Schema

Student (Student_ID, Name, Major, Class)

- Student Entity keeps track of the students primarily through their Student_ID (sort of like a TigerID)

Faculty (Faulty_ID, Name, Department)

- Faculty Entity is primarily identifiable through their Faculty_ID, they will also be an advisor role for the different clubs

Club (Club_ID, Club Name, Description)

- Clubs will be primarily identifiable through their Club ID which can also be used to distinguish the different students, rooms and events via the clubs

Membership (Student_ID, Member_ID, Club_ID, Role)

- Membership will be an entity that shows which students are a part of which clubs on campus

Member_Event (Event_ID, Club_ID, Member_ID, Room_Number, Activity)

- Member Event will be differentiated by the public events via their Member IDs which are assigned to students via the Member IDs in the Membership Entity

Public_Event (Event_ID, Club_ID, Room_Number, Activity)

- These public events will be public and “open to all” meaning there is no requirement for membership in order for people to participate in these club events

Meeting_Room (Room_Number, Building, Capacity

- Meeting Rooms will be identifiable through their room numbers and buildings, this helps prevent overlapping in clubs hosting club events (whether public or members only) on the same rooms

Data Generation

- Drops tables if they already exist due to a prior run through
- Creates Student table
- Creates Faculty table
- Creates Club table
- Creates Membership table
- Creates Member_Event table
- Creates Public_Event table
- Creates Meeting_Room table
- Inserts values into Student
- Inserts values into Faculty
- Inserts values into Club
- Inserts values into Membership
- Inserts values into Member_Event
- Inserts values into Public_Event
- Inserts values into Meeting_Room

Test Queries

~~Below are five test queries that were chosen to test different parts of the database. Each query has a short explanation of what it verifies, the SQL code for the query itself, and a small example of query output from our data.~~

- Query 1
    - Checks that Student, Membership, and Club all join correctly, and that the role stored in Membership shows up with the correct student and club.
    - SQL:
        - ```
          SELECT
                s.Student_ID,
                s.Name AS StudentName,
                c.Club_ID,
                c.Club_Name AS ClubName,
                m.Role
          FROM Membership AS m
          JOIN Student AS s ON m.Student_ID = s.Student_ID
          JOIN Club AS c ON m.Club_ID = c.Club_ID
          ORDER BY s.Student_ID, c.Club_ID;
          ```
    - Sample output:

Student_ID	StudentName	Club_ID	ClubName	Role
89450	Jordan	2	Board Game Club	Member
89555	Robert	3	SASE	President
89460	Shaquille	4	ClubA	Vice President
          ```

89775	Alice	1	Robotics	Treasurer
89651	Cassandra	5	ClubB	Member

- Query 2
  - Tests the Membership to Club connection and makes sure aggregation works correctly
  - SQL:
    - SELECT
      c.Club_ID,
      c.Club_Name,
      COUNT(m.Member_ID) AS MemberCount
      FROM Club AS c
      LEFT JOIN Membership AS m
      ON c.Club_ID = m.Club_ID
      GROUP BY c.Club_ID, c.Club_Name
      ORDER BY c.Club_ID;
  - Sample output:

Club_ID	Club_Name	MemberCount
1	Robotics	1
2	Board Game Club	1
3	SASE	1
4	ClubA	1
5	ClubB	1

- Query 3
  - Checks the long join chain: Member_Event to Membership to Student to Club to Room. If it runs without issues, the schema is behaving as expected.
  - SQL:
    - SELECT
      me.Event_ID,
      c.Club_Name,
      s.Student_ID,
      s.Name AS StudentName,
      me.Activity,
      me.Room_Number
      FROM Member_Event AS me
      JOIN Membership AS m ON me.Member_ID = m.Member_ID
      JOIN Student AS s ON m.Student_ID = s.Student_ID
      JOIN Club AS c ON me.Club_ID = c.Club_ID
      ORDER BY me.Event_ID;
  - Sample output:

Event_ID	Club_Name	Student_ID	StudentName	Activity	Room_Number
44000	SASE	89555	Robert	Feeding other ducks	43
45000	Robotics	89775	Alice	Something important	55
46000	Board Game Club	89450	Jordan	Feeding ducks	25
47000	ClubA	89460	Shaquille	Feeding geese	44

48000	ClubB	89651	Cassandra	Feeding geese to ducks	87

- Query 4
  - Checks filtering and the join between Public_Event and Club.
  - SQL:
    - SELECT
      pe.Event_ID,
      c.Club_Name,
      pe.Activity,
      pe.Room_Number
      FROM Public_Event AS pe
      JOIN Club AS c ON pe.Club_ID = c.Club_ID
      WHERE pe.Activity LIKE '%duck%'
      ORDER BY pe.Event_ID;
  - Sample output:

Event_ID	Club_Name	Activity	Room_Number
41000	SASE	Something with ducks	43

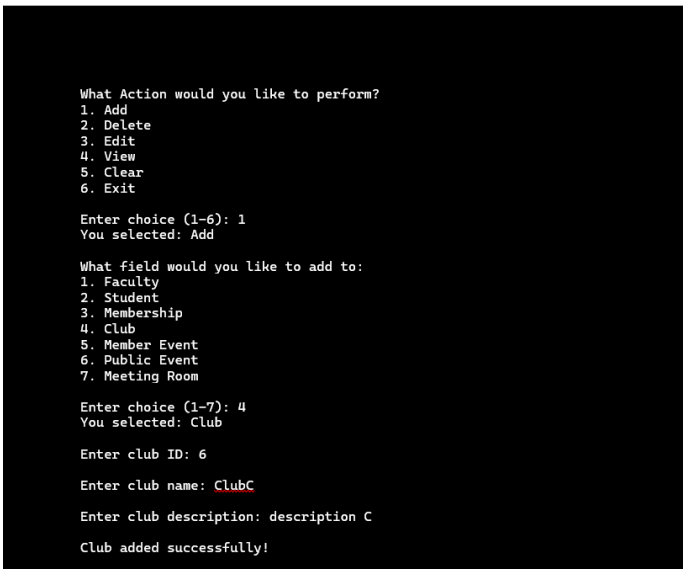
- Query 5
  - Tests both event tables are connected to Club correctly. Also shows the database can aggregate over multiple joined tables.
  - SQL:
    - SELECT
      c.Club_ID,
      c.Club_Name,
      COUNT(DISTINCT me.Event_ID) AS MemberEvents,
      COUNT(DISTINCT pe.Event_ID) AS PublicEvents,
      COUNT(DISTINCT me.Event_ID) + COUNT(DISTINCT pe.Event_ID) AS TotalEvents
      FROM Club AS c
      LEFT JOIN Member_Event AS me ON c.Club_ID = me.Club_ID
      LEFT JOIN Public_Event AS pe ON c.Club_ID = pe.Club_ID
      GROUP BY c.Club_ID, c.Club_Name
      ORDER BY c.Club_ID;
  - Sample output:

Club_ID	Club_Name	MemberEvents	PublicEvents	TotalEvents
1	Robotics	1	1	2
2	Board Game Club	1	1	2
3	SASE	1	1	2
4	ClubA	1	1	2
5	ClubB	1	1	2

Application User Interfaces

For the Application UI, we went with a simple CLI that allows the user to perform 4 commands to add, edit, delete, and view. The add command will essentially be like using INSERT INTO, the edit command will use UPDATE, SET, and WHERE in sequence, delete will use DELETE FROM, and WHERE in sequence, and view will SELECT all items from a table and display them. All CLI command prompts were illustrated on word docs to get an idea of what text prompts we wanted.

Example of add:

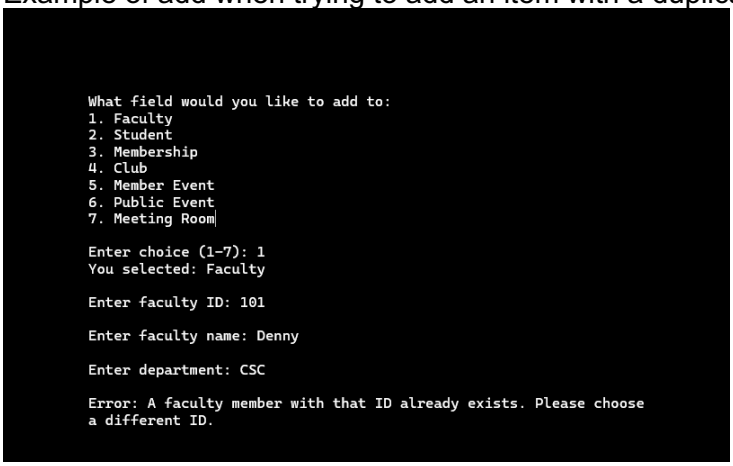


This example would be the equivalent of running

INSERT INTO Club (Club_ID, Club_Name, Description) VALUES

(6, ‘ClubC’, ‘description C’)

Example of add when trying to add an item with a duplicate super key:



Example of View:

```
What would you like to view:
1. Faculty Members
2. Students
3. Memberships
4. Clubs
5. Member Events
6. Public Events
7. Meeting Rooms
8. List Students in a Club
9. List Clubs a student is in

Enter choice (1-9): 2
You selected: Student

Students:
ID: 89450, Name: Jordan, Major: Computer Science, Class: Compiler
Construction
ID: 89555, Name: Robert, Major: Mechanical Engineering, Class:
Thermodynamics
ID: 89460, Name: Shaquille, Major: Chemical Engineering, Class:
CHEM4000
ID: 89775, Name: Alice, Major: Computer Science, Class: Database
Design
ID: 89651, Name: Cassandra, Major: Electrical Engineering, Class:
Circuit Design
```

This example would the SQL equivalent to SELECT * Student

Example of delete:

```
What field would you like to delete from:

1. Faculty
2. Student
3. Membership
4. Club
5. Member Event
6. Public Event
7. Meeting room

Enter choice (1-7):

4

Enter Club ID to delete:

1

Delete Club : Club_ID =1, Club_Name=Robotics, Description=Makes robots

Are you sure? (yes/no)

yes

Club deleted successfully
```